

01. Context manager: entry, exit, and exception suppression

PREDICT OUTPUT + EXPLAIN

```
class Session:
    def __enter__(self):
        print("enter")
        return "ready"

    def __exit__(self, exc_type, exc, tb):
        print("exit", exc_type.__name__ if exc_type else None)
        return True

print("before")
with Session() as value:
    print("body", value)
    raise ValueError("bad")
    print("unreachable")
print("after")
```

QUESTIONS

1. Predict the exact printed output and explain what is assigned to value.
2. What changes if `__exit__` returns False? Does the final print run?
3. If `__enter__` itself raises, is `__exit__` called? Explain how a context manager supports resource cleanup.

COMPACT ANSWERS

1. Exact output:

```
before
enter
body ready
exit ValueError
after
```

value is "ready", the return from `__enter__`. Raising skips "unreachable". Returning True from `__exit__` suppresses the error.

2. With False, the first four lines are unchanged. `ValueError("bad")` propagates; the unhandled error prevents "after" from printing.
3. If `__enter__` raises, `__exit__` is not called and the body never starts. After successful entry, `__exit__` performs cleanup on normal exit or a body exception. Partial setup failures must clean up inside `__enter__`.

02. Stacked decorators: decoration time versus call time

PREDICT OUTPUT + EXPLAIN

```
def outer(func):
    print("apply outer")
    def wrapper():
        print("outer before")
        result = func()
        print("outer after")
        return result
    return wrapper

def inner(func):
    print("apply inner")
    def wrapper():
        print("inner before")
        result = func()
        print("inner after")
        return result
    return wrapper

@outer
@inner
def job():
    print("job")
    return 7

print("call")
print(job())
```

QUESTIONS

1. Predict every printed line, including lines printed when job is defined.
2. Rewrite the decorator syntax as a single assignment without @.
3. Explain the order of decorator application and the order of wrapper execution.

COMPACT ANSWERS

1. Exact output:

```
apply inner
apply outer
call
outer before
inner before
job
inner after
outer after
7
```

2. Keep the original def job block, remove both @ lines, and add this immediately after the definition:

```
job = outer(inner(job))
```

3. Application: inner first, then outer, while job is being defined, before print("call"). A decorator returns a wrapper without running it. Call: enter outer, enter inner, run job; then resume inner and outer in reverse order. The return value 7 passes through both wrappers.

03. __new__ versus __init__: debug a singleton

DEBUG + EXPLAIN

```
class Connection:
    _instance = None

    def __new__(cls, label):
        print("new", label)
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            # A required statement is missing.

    def __init__(self, label):
        print("init", label)
        self.label = label

a = Connection("first")
b = Connection("second")
print(a is b)
print(a)
```

QUESTIONS

1. Predict the printed output of this broken program and identify the missing statement.
2. After fixing __new__, what would print(a.label, b.label) show? Why?
3. Modify the single-threaded implementation so the shared instance is initialized only once.
4. Explain the object-creation lifecycle and give another reason to override __new__.

COMPACT ANSWERS

1. Broken output:

```
new first
new second
True
None
```

Missing: return cls._instance, outside the if. Without it __new__ returns None, so a and b are None and __init__ never runs.

2. After that fix: second second. Both names refer to one instance; the second construction calls __init__ again and overwrites its label.

3. Keep the corrected __new__; replace __init__ with:

```
def __init__(self, label):
    if getattr(self, "_initialized", False):
        return
    print("init", label)
    self.label = label
    self._initialized = True
```

Now both labels stay "first". __init__ is still called, but later calls return before initialization.

4. Construction calls __new__ to obtain an object, then __init__ if the result is an instance of the requested class. The object is returned to the caller. __init__ must return None. Also override __new__ to set the value of an immutable subclass, such as str or int, during creation.

04. async with: implement asynchronous resource cleanup

IMPLEMENT + TRACE

```
import asyncio

class AsyncConnection:
    def __init__(self):
        self.open = False

    async def __aenter__(self):
        pass # Implement.

    async def __aexit__(self, exc_type, exc, tb):
        pass # Implement.

async def main():
    try:
        async with AsyncConnection() as conn:
            print(conn.open)
            raise RuntimeError("failure")
    except RuntimeError:
        print("caught")

asyncio.run(main())
```

QUESTIONS

1. Implement entry: await asyncio.sleep(0), set open to True, print 'opened', and return the connection.
2. Implement exit: await asyncio.sleep(0), set open to False, print 'closed', and allow exceptions to propagate.
3. Predict the completed program's output. Why is async with useful when acquisition or cleanup needs await?

COMPACT ANSWERS

1-2. Replace the two placeholder methods inside AsyncConnection:

```
async def __aenter__(self):
    await asyncio.sleep(0)
    self.open = True
    print("opened")
    return self

async def __aexit__(self, exc_type, exc, tb):
    await asyncio.sleep(0)
    self.open = False
    print("closed")
    return False
```

3. Completed output:

```
opened
True
closed
caught
```

async with awaits setup and cleanup. return self assigns the connection to conn. The body error triggers cleanup; False lets the error reach except. Async I/O can yield to other tasks, while this task waits for cleanup before its handler runs.

05. async for: implement an asynchronous iterator

IMPLEMENT + EXPLAIN

```
import asyncio

class AsyncRange:
    def __init__(self, stop):
        self.current = 0
        self.stop = stop

    def __aiter__(self):
        pass # Implement.

    async def __anext__(self):
        pass # Implement.

async def main():
    async for value in AsyncRange(3):
        print(value)

asyncio.run(main())
```

QUESTIONS

1. Implement an iterator yielding integers from 0 up to, but excluding, a non-negative stop.
2. Await `asyncio.sleep(0)` before returning each value; terminate using the appropriate exception.
3. Explain why `__anext__` is asynchronous and whether `__aiter__` should be `async def` here.

COMPACT ANSWERS

1-2. Replace the methods inside `AsyncRange`:

```
def __aiter__(self):
    return self

async def __anext__(self):
    if self.current >= self.stop:
        raise StopAsyncIteration
    await asyncio.sleep(0)
    value = self.current
    self.current += 1
    return value
```

Output: 0, 1, 2 on separate lines. `stop=0` produces no values.

3. `__anext__` is `async` because obtaining the next item can require `await`. Here `__aiter__` must be an ordinary `def` that returns the asynchronous iterator directly, rather than a coroutine. `StopAsyncIteration` tells `async for` to finish.

06. typing.Protocol: structural interfaces

EXPLAIN + DIAGNOSE

```
from typing import Protocol

class DocumentSource(Protocol):
    def fetch(self, doc_id: str) -> str: ...

class MemorySource:
    def fetch(self, doc_id: str) -> str:
        return f"document:{doc_id}"

class BrokenSource:
    def fetch(self, doc_id: str) -> int:
        return 123

def read_document(source: DocumentSource) -> str:
    return source.fetch("A")

print(read_document(MemorySource()))
```

QUESTIONS

1. Why can MemorySource satisfy DocumentSource without inheriting from it?
2. Would a static type checker accept BrokenSource? Does the annotation automatically enforce the return type at runtime?
3. What happens with isinstance(MemorySource(), DocumentSource)? How does @runtime_checkable change the check, and what does it fail to validate?
4. Give a situation where a Protocol would help you design interchangeable components.

COMPACT ANSWERS

1. Protocol uses structural typing: MemorySource has a compatible fetch(str) -> str method, so inheritance is unnecessary. The shown print produces document:A.
2. A static checker rejects BrokenSource when supplied where DocumentSource is expected: fetch returns int instead of str. Annotations do not enforce this at runtime; read_document(BrokenSource()) returns 123.
3. The original isinstance call raises TypeError. Decorate the Protocol class as follows:

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class DocumentSource(Protocol):
    def fetch(self, doc_id: str) -> str: ...
```

Now isinstance(MemorySource(), DocumentSource) is True. The runtime check checks member presence, not method signatures or return types; BrokenSource also passes it.

4. A retrieval service can accept a DocumentSource implemented by local files, a database, or a test fake. Static checks verify the shared interface without forcing a common implementation.

07. Parameterized decorator: write a decorator factory

IMPLEMENT

```
def announce(prefix):
    pass # Implement the decorator factory.

@announce("TRACE")
def add(a, b=0):
    return a + b

print(add(2, b=3))
```

QUESTIONS

1. Implement announce so each function call prints '<prefix>:<function name>' before calling the function exactly once.
2. Support any positional and keyword arguments and return the original function's result.
3. Explain the three layers: factory, decorator, and wrapper.

COMPACT ANSWERS

1-2. Implementation (wraps also preserves metadata):

```
from functools import wraps

def announce(prefix):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            print(f"{prefix}:{func.__name__}")
            return func(*args, **kwargs)
        return wrapper
    return decorator
```

The original decorated add and print remain unchanged. Output:

```
TRACE:add
5
```

3. Factory: announce("TRACE") stores the prefix and returns a decorator. Decorator: accepts add and returns its wrapper during definition. Wrapper: runs on each add(...) call, prints once, forwards all arguments, calls add exactly once, and returns its result.

08. functools.wraps: preserve function metadata

PREDICT OUTPUT + FIX

```
def traced(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@traced
def total(a, b):
    """Add two numbers."""
    return a + b

print(total.__name__)
print(total.__doc__)
```

QUESTIONS

1. Predict the exact output before making any changes.
2. Add functools.wraps in the correct place and predict the new output.
3. What is `__wrapped__`, and how does preserved metadata help documentation, debugging, or introspection?

COMPACT ANSWERS

1. Original output:

```
wrapper
None
```

The name `total` now points to `wrapper`, which has no docstring.

2. Replace `traced` with:

```
from functools import wraps

def traced(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

New output:

```
total
Add two numbers.
```

3. `total.__wrapped__` points to the original function. `wraps` preserves attributes such as `__name__`, `__doc__`, and `__annotations__`, and sets `__wrapped__`. Documentation, debugging, and tools such as `inspect.signature` can discover the original function and signature.

09. GIL: reason about CPU-bound threads

EXPLAIN

```
def work(n):  
    total = 0  
    for i in range(n):  
        total += i * i  
    return total
```

QUESTIONS

1. Assume conventional CPython with its GIL enabled. What does the GIL restrict?
2. Would four threads running four equally sized work tasks guarantee a speedup over running those same tasks sequentially? Explain.
3. Why can threads still help with I/O? Can native extensions behave differently from this pure-Python loop?
4. Does the GIL guarantee that an arbitrary multi-step update to shared data is race-free? What changes in a free-threaded CPython build?

COMPACT ANSWERS

1. With the GIL enabled, one thread at a time executes Python bytecode in a given interpreter.
2. No guaranteed speedup. This loop is CPU-bound Python work, so four threads share that execution capacity; scheduling can add overhead. Processes can run independent jobs on separate CPU cores.
3. Threads help with I/O because blocking I/O commonly releases the GIL. Native extensions can release it during computation and run in parallel; this depends on the extension.
4. The GIL does not make arbitrary read-modify-write sequences safe. Use a lock for shared multi-step state changes. Optional free-threaded CPython builds, introduced in 3.13, can execute Python threads in parallel when the GIL is disabled. Shared application invariants still need synchronization; some extensions can re-enable the GIL.

10. Multithreading versus multiprocessing: choose an execution model

COMPARE + DESIGN

QUESTIONS

1. System A downloads 200 documents and spends most of its time waiting for responses. System B performs independent, expensive pure-Python computations on 200 documents. Choose `ThreadPoolExecutor` or `ProcessPoolExecutor` for each, and justify the choice.
2. Compare memory sharing, startup overhead, argument serialization, and communication between workers.
3. If a worker appends to an ordinary list, when does the parent see the change: a thread or a separate process? How would you return results from process workers?

COMPACT ANSWERS

1. A: `ThreadPoolExecutor`, because network waits dominate. B: `ProcessPoolExecutor`, because independent CPU-heavy Python jobs can use multiple cores despite an enabled GIL.

2. Comparison:

Aspect	Threads	Processes
Memory	Share objects/address space	Separate address spaces
Startup	Usually lower overhead	Usually higher overhead
Arguments	Object references; no pickle	Pool arguments/results need pickling
Communication	Shared state; locks/queues	Returned results, IPC/queues

3. A thread appending to a shared ordinary list changes the same list the parent holds; wait for completion to inspect results reliably. A separate process changes its own ordinary list, not the parent's. Return data from the worker and collect `future.result()` or `executor.map(...)`. Use a process-safe queue or explicit shared storage when needed.

11. Lookup complexity: indexing, membership, and hashing

ANALYZE COMPLEXITY

QUESTIONS

1. Assume `values` is a list, `lookup` is a dictionary, and `unique` is a set, each containing n items. Assume constant-cost hash and equality operations and a valid list index.
2. Give average-case and worst-case time complexity for: `values[i]`; `target in values`; `key in lookup`; `lookup[key]`; `target in unique`; `target in lookup.values()`.
3. Explain why list indexing and list membership differ, and why dictionary/set lookup is described as average $O(1)$.

COMPACT ANSWERS

1. Under the stated assumptions, the operations have these costs:

Operation	Average	Worst
<code>values[i]</code>	$O(1)$	$O(1)$
<code>target in values</code>	$O(n)$	$O(n)$
<code>key in lookup</code>	$O(1)$	$O(n)$
<code>lookup[key]</code>	$O(1)$	$O(n)$
<code>target in unique</code>	$O(1)$	$O(n)$
<code>target in lookup.values()</code>	$O(n)$	$O(n)$

2. The table gives both requested cases. A missing dictionary key raises `KeyError` with the same lookup cost.
3. List indexing jumps to a known position. Membership searches by value and can scan all n elements. Dictionaries and sets use hashes to locate candidates, giving average $O(1)$; severe collisions can require $O(n)$. Dictionary values are not indexed by value and must be scanned.

12. List comprehension: flatten, filter, and transform

PREDICT OUTPUT + REWRITE

```
rows = [[1, 2, 3], [4], [], [5, 6]]
result = [
    n * n
    for row in rows
    for n in row
    if n % 2 == 0
]
print(result)
```

QUESTIONS

1. Predict the exact output and explain the order of the two for clauses.
2. Rewrite the comprehension as ordinary nested loops.
3. Change the comprehension to square only odd numbers greater than 2, preserving their original traversal order.

COMPACT ANSWERS

1. Output: [4, 16, 36]. Traverse each row first, then each n within that row; keep even n and square it.
2. Equivalent loops:

```
result = []
for row in rows:
    for n in row:
        if n % 2 == 0:
            result.append(n * n)
print(result)
```

3. Odd numbers greater than 2:

```
result = [n * n for row in rows for n in row
          if n % 2 != 0 and n > 2]
print(result) # [9, 25]
```

13. Dictionary comprehension: duplicate keys

PREDICT OUTPUT + MODIFY

```
pairs = [("A", 2), ("B", 3), ("A", 5)]
result = {key: value * 2 for key, value in pairs}
print(result)
```

QUESTIONS

1. Predict the exact output. What happens when a key occurs more than once?
2. Modify the comprehension to include only pairs whose value is at least 3, while retaining the multiplication by 2.
3. Rewrite the comprehension as an ordinary loop, then describe how you would retain all repeated-key values as lists instead.

COMPACT ANSWERS

1. Output: {'A': 10, 'B': 6}. The last value for A replaces the earlier value. Updating A does not move its original insertion position.

2. Filter before inserting:

```
result = {key: value * 2 for key, value in pairs if value >= 3}
print(result) # {'B': 6, 'A': 10}
```

3. Equivalent to the original comprehension:

```
result = {}
for key, value in pairs:
    result[key] = value * 2
```

To retain every doubled value, group them into lists:

```
grouped = {}
for key, value in pairs:
    grouped.setdefault(key, []).append(value * 2)
print(grouped) # {'A': [4, 10], 'B': [6]}
```

Use `append(value)` instead to retain the original, undoubled values.

14. map, filter, and reduce: functional data processing

PREDICT OUTPUT + REWRITE

```
from functools import reduce
values = [1, 2, 3, 4, 5]
mapped = map(lambda n: n * 2, values)
selected = filter(lambda n: n > 5, mapped)

print(reduce(lambda total, n: total + n, selected, 0))
print(list(mapped))
print(list(selected))
```

QUESTIONS

1. Predict the exact output and explain transformation, selection, and aggregation separately.
2. Which operations create lazy iterators, and which operation consumes them here?
3. Rewrite the whole transformation/filtering/summing pipeline with an ordinary loop.
4. What happens when reduce receives an empty iterable with, and without, an initializer?

COMPACT ANSWERS

1. Exact output:

```
24
[]
[]
```

map doubles to 2, 4, 6, 8, 10; filter keeps 6, 8, 10; reduce sums these to 24. Both iterators are exhausted afterward.

2. map and filter are lazy iterators. reduce pulls from selected, which pulls from mapped; one consumption exhausts the whole pipeline.

3. Equivalent calculation with an ordinary loop:

```
total = 0
for n in values:
    doubled = n * 2
    if doubled > 5:
        total += doubled
print(total) # 24
```

4. reduce on an empty iterable returns its initializer, if supplied (0 here). Without an initializer it raises TypeError.

15. Iterator protocol: `__iter__`, `__next__`, and exhaustion

PREDICT OUTPUT + EXPLAIN

```
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        value = self.current
        self.current -= 1
        return value

counter = Countdown(3)
print(next(counter))
print(list(counter))
print(list(counter))
```

QUESTIONS

1. Predict the exact output.
2. Explain the difference between an iterable and an iterator. Is an ordinary list itself an iterator?
3. Why does this object not restart on a second traversal? Describe a design in which `__iter__` returns a fresh iterator.

COMPACT ANSWERS

1. Exact output:

```
3
[2, 1]
[]
```

2. An iterable supplies an iterator through `iter(obj)`. An iterator supplies successive values through `next(obj)`, returns itself from `iter(obj)`, and raises `StopIteration` when exhausted. A list is iterable, but is not itself an iterator; `iter(a_list)` creates one.
3. `Countdown` stores one advancing current value and returns itself, so it stays exhausted. For repeatable traversal, store the starting value and create a fresh iterator each time:

```
class RepeatableCountdown:
    def __init__(self, start):
        self.start = start

    def __iter__(self):
        return iter(range(self.start, 0, -1))
```

16. Generator: lazy execution and suspended state

PREDICT OUTPUT + EXPLAIN

```
def scores():
    print("start")
    yield 10
    print("middle")
    yield 20
    print("end")

g = scores()
print("created")
print(next(g))
print(list(g))
print(list(g))
```

QUESTIONS

1. Predict every printed line and identify when the function body begins executing.
2. Explain how yield differs from return and how the function resumes after a yield.
3. How is a generator related to an iterator? Why might a generator help when processing a very large file?

COMPACT ANSWERS

1. Exact output:

```
created
start
10
middle
end
[20]
[]
```

scores() creates a generator; its body starts at next(g). list(g) resumes it, collects 20, and runs through "end" before printing [20].

2. yield emits one value and pauses, preserving local state and the execution position. The next request resumes after that yield. return ends the generator; further requests raise StopIteration.
3. A generator is an iterator and is normally consumed once. Yielding file records on demand avoids building a list of the entire file in memory.

17. finally: cleanup and return precedence

PREDICT OUTPUT + EXPLAIN

```
def evaluate():  
    try:  
        print("try")  
        return "from try"  
    finally:  
        print("finally")  
        return "from finally"  
  
print(evaluate())
```

QUESTIONS

1. Predict the exact output.
2. What changes if the return statement inside finally is removed?
3. Under normal interpreter execution, explain when finally runs after successful completion, return, or an exception.
4. If try raises an exception but finally returns a value, what happens to the exception?

COMPACT ANSWERS

1. Exact output:

```
try  
finally  
from finally
```

2. Remove the return in finally and the output becomes:

```
try  
finally  
from try
```

3. Under normal execution, finally runs when control leaves try: on success, a pending return, or an exception. After cleanup, the pending return or exception continues unless finally replaces it.
4. A return in finally suppresses a pending exception and returns its own value. It can also replace an earlier return. Keep cleanup there, and normally avoid returning from finally so errors are preserved.

18. Exception handlers: broad versus specific exceptions

PREDICT OUTPUT + FIX

```
values = ["10", "bad", None]
for value in values:
    try:
        print(int(value))
    except Exception:
        print("general")
    except ValueError:
        print("invalid number")
    except TypeError:
        print("unsupported type")
```

QUESTIONS

1. Predict the exact output and explain why some handlers never run.
2. Reorder the handlers so conversion and unsupported-type errors receive their intended messages.
3. Does `except Exception` catch `KeyboardInterrupt` or `SystemExit`? Explain the relevant exception hierarchy.

COMPACT ANSWERS

1. Original output:

```
10
general
general
```

`ValueError` and `TypeError` are subclasses of `Exception`. The first matching handler wins, so the broad handler shadows both specific ones.

2. Corrected loop:

```
for value in values:
    try:
        print(int(value))
    except ValueError:
        print("invalid number")
    except TypeError:
        print("unsupported type")
    except Exception:
        print("general")
```

Corrected output:

```
10
invalid number
unsupported type
```

3. `except Exception` does not catch `KeyboardInterrupt` or `SystemExit`. Those inherit directly from `BaseException`, which is also the parent of `Exception`.

19. Custom exception: validate domain rules

IMPLEMENT

QUESTIONS

1. Define `InvalidScoreError` as a subclass of `ValueError`.
2. Implement `validate_score(value)`: accept integers from 0 through 100 inclusive, reject booleans explicitly, raise `InvalidScoreError` with a useful message for every invalid input, and return the accepted score.
3. Write a caller that processes `[50, True, -1, 101, '90']`, catches `InvalidScoreError` for each rejected item, and continues to the next item.
4. Explain why a custom exception helps callers distinguish this domain failure from other errors.

COMPACT ANSWERS

1-3. Define the domain exception, validate, and handle each item independently:

```
class InvalidScoreError(ValueError):
    pass

def validate_score(value):
    if isinstance(value, bool) or not isinstance(value, int):
        raise InvalidScoreError(
            f"Expected an integer score; got {value!r}")
    if not 0 <= value <= 100:
        raise InvalidScoreError(
            f"Score must be from 0 to 100; got {value}")
    return value

for value in [50, True, -1, 101, '90']:
    try:
        print(validate_score(value))
    except InvalidScoreError as exc:
        print(f"rejected: {exc}")
```

50 is accepted; all four remaining inputs are rejected and processing continues. 0 and 100 are valid. Reject `bool` explicitly because `bool` is a subclass of `int`.

4. A caller can catch this specific domain failure without confusing it with unrelated `ValueErrors`. Subclassing `ValueError` also lets a broader `ValueError` handler catch it when desired.

20. Classes and objects: class versus instance attributes

PREDICT OUTPUT + FIX

```
class Player:
    tags = []

    def __init__(self, name):
        self.name = name

a = Player("A")
b = Player("B")
a.tags.append("winner")
b.name = "C"

print(a.name, b.name)
print(a.tags, b.tags)
```

QUESTIONS

1. Predict the exact output.
2. Explain why the names are independent but the tags are shared.
3. Fix the class so every player has its own tags list. Explain the roles of the class, each object, self, and __init__.

COMPACT ANSWERS

1. Exact output:

```
A C
['winner'] ['winner']
```

2. self.name is stored separately on each instance. tags is one class-level list; both instances find that same list, and append mutates it.
3. Remove the class-level tags list and initialize a fresh one per player:

```
class Player:
    def __init__(self, name):
        self.name = name
        self.tags = []
```

With the original caller, output becomes A C, then ['winner'] [] on the next line. The class defines behavior; a and b are distinct objects. self identifies the object receiving the method call. __init__ initializes that object's attributes.

21. Inheritance: initialize the base class with super

DEBUG + EXPLAIN

```
class User:
    def __init__(self, name):
        self.name = name
        print("User init")

class Admin(User):
    def __init__(self, name, level):
        self.level = level
        print("Admin init")

admin = Admin("Jay", 2)
print(admin.name, admin.level)
```

QUESTIONS

1. State what is printed before the program fails, and identify the exception type.
2. Fix Admin so it initializes both inherited and subclass attributes; call the base initializer first.
3. Predict the corrected output and explain why the base initializer was not automatically run in the original code.

COMPACT ANSWERS

1. Prints Admin init, then raises AttributeError while accessing admin.name; User.__init__ never set it.
2. Replace Admin with:

```
class Admin(User):
    def __init__(self, name, level):
        super().__init__(name)
        self.level = level
        print("Admin init")
```

3. Corrected output:

```
User init
Admin init
Jay 2
```

Admin overrides __init__, so its method is used. Python does not automatically chain all base initializers. super().__init__(name) explicitly calls the next initializer in the method resolution order; here that is User.__init__.

22. Polymorphism: one operation, interchangeable objects

PREDICT OUTPUT + EXPLAIN

```
class EmailSender:
    def send(self, message):
        return f"email:{message}"

class SmsSender:
    def send(self, message):
        return f"sms:{message}"

def notify(sender, message):
    return sender.send(message)

for sender in [EmailSender(), SmsSender()]:
    print(notify(sender, "hello"))
```

QUESTIONS

1. Predict the exact output and explain why notify works with both unrelated classes.
2. Add a PushSender without changing notify.
3. Does this form of runtime polymorphism require a common base class? What happens if an object lacks send?

COMPACT ANSWERS

1. Exact output:

```
email:hello
sms:hello
```

notify needs an object with a compatible send method. Each sender supplies its own implementation: this is duck typing.

2. Add a third interchangeable implementation:

```
class PushSender:
    def send(self, message):
        return f"push:{message}"

print(notify(PushSender(), "hello")) # push:hello
```

3. A common base class is unnecessary. An object without send raises AttributeError when notify tries to access it. A non-callable send attribute or incompatible signature can instead raise TypeError.

23. Encapsulation: properties and name mangling

PREDICT OUTPUT + IMPLEMENT

```
class Account:
    def __init__(self):
        self.__balance = 100

    @property
    def balance(self):
        return self.__balance

account = Account()
account.__balance = 999
print(account.balance)
print(account.__balance)
```

QUESTIONS

1. Predict the exact output. Did the external assignment modify the attribute used by the property?
2. Explain double-underscore name mangling and whether it provides strict privacy.
3. What happens if you execute `account.balance = -50`?
4. Add a `withdraw(amount)` method that rejects non-positive amounts and overdrafts, while preserving a non-negative balance. Assume `amount` is an integer.

COMPACT ANSWERS

1. Exact output:

```
100
999
```

The external assignment creates `__balance`. The property still reads the separate, mangled `_Account__balance` attribute.

2. Within the class, `__balance` is name-mangled to `_Account__balance`, reducing accidental clashes, especially in subclasses. It is not strict privacy: callers can still access the mangled name.
3. `account.balance = -50` raises `AttributeError: this property has no setter`.
4. Add this method inside `Account`; keep the existing property:

```
def withdraw(self, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive")
    if amount > self.__balance:
        raise ValueError("Insufficient balance")
    self.__balance -= amount
    return self.__balance
```

Both checks occur before mutation, so rejected withdrawals leave the balance unchanged. With the stated integer-input assumption, withdrawing the full balance leaves 0.

24. Method overloading versus overriding

DEBUG + COMPARE

```
class Formatter:
    def format(self, value):
        return f"one:{value}"

    def format(self, value, prefix):
        return f"{prefix}:{value}"

formatter = Formatter()
print(formatter.format("hello"))
```

QUESTIONS

1. Identify the exception and explain why two declarations did not create two runtime overloads.
2. Replace them with one method that supports both `format('hello')` and `format('hello', 'tag')`.
3. Explain how overriding a method in a subclass differs from this attempted overloading.
4. Does `typing.overload` alone implement runtime argument dispatch?

COMPACT ANSWERS

1. `TypeError`: the active `format` method requires `prefix`. The second `def` replaces the first under the same class attribute name; Python does not select methods by argument count.

2. Use one method with a default parameter:

```
class Formatter:
    def format(self, value, prefix="one"):
        return f"{prefix}:{value}"

formatter = Formatter()
print(formatter.format("hello"))          # one:hello
print(formatter.format("hello", "tag"))   # tag:hello
```

3. Overriding defines a replacement method in a subclass; lookup uses the object's class and method resolution order. The base method remains available, for example through `super()`. Overloading describes different signatures for one operation.

4. `typing.overload` only describes signatures for static checkers. Supply a real implementation for runtime calls; it does not automatically dispatch arguments.

25. staticmethod versus classmethod: subclass-aware factories

PREDICT OUTPUT + EXPLAIN

```
class Player:
    def __init__(self, name):
        self.name = name

    @classmethod
    def from_record(cls, record):
        return cls(record["name"])

    @staticmethod
    def valid_name(name):
        return bool(name.strip())

class VipPlayer(Player):
    pass

player = VipPlayer.from_record({"name": "Jay"})
print(type(player).__name__, player.name)
print(VipPlayer.valid_name(" "))
```

QUESTIONS

1. Predict the exact output.
2. Explain what is passed automatically to an instance method, a classmethod, and a staticmethod.
3. What changes if from_record returns Player(record['name']) instead of cls(record['name'])?
4. Why is a classmethod useful for an alternative constructor?

COMPACT ANSWERS

1. Exact output:

```
VipPlayer Jay
False
```

2. Bound instance method: receives self. classmethod: receives the calling class as cls (VipPlayer here). staticmethod: receives neither automatically; it uses only explicitly supplied arguments.
3. Returning Player(record["name"]) hard-codes the base type, so the first output becomes Player Jay. The valid_name result stays False.
4. A classmethod supports an alternative constructor, such as parsing a record. Calling cls(...) creates the appropriate subclass when the factory is inherited and its constructor has a compatible signature.

26. Collection types: list, tuple, set, and dictionary

PREDICT OUTPUT + COMPARE

```
values = [3, 1, 3, 2]
as_tuple = tuple(values)
as_set = set(values)
as_dict = {value: value * 10 for value in values}

print(len(values), len(as_tuple), len(as_set), len(as_dict))
print(list(as_dict))
```

QUESTIONS

1. Predict the exact output; do not assume any iteration order for a set.
2. Compare the four types on ordering, duplicates, positional indexing, and mutability.
3. Choose a type for each: an ordered editable task list; a fixed coordinate pair; unique player IDs; player-name lookup by ID.
4. What requirement must a set element or dictionary key satisfy?

COMPACT ANSWERS

1. Exact output:

```
4 4 3 3
[3, 1, 2]
```

A repeated dictionary key updates its value without moving its original insertion position.

2. Comparison:

Type	Order	Duplicates	Position index	Mutable
list	Preserved	Allowed	Yes	Yes
tuple	Preserved	Allowed	Yes	No
set	No guarantee	Unique elements	No	Yes
dict	Insertion order	Unique keys	No; key lookup	Yes

Dictionary values may repeat; only keys are unique. Tuple immutability fixes its references, not the internal state of objects it references.

3. Editable ordered tasks: list. Fixed coordinate pair: tuple. Unique player IDs: set. Name lookup by player ID: dict.
4. Set elements and dictionary keys must be hashable: a stable hash while stored, with equality-compatible hashing (equal objects have equal hashes). A tuple is hashable only if all its elements are hashable.

27. Mutable versus immutable: mutation and rebinding

PREDICT OUTPUT + EXPLAIN

```
def change(items, count):
    items += [3]
    count += 1
    return count

numbers = [1, 2]
count = 0
returned = change(numbers, count)
print(numbers, count, returned)

pair = ([10], 20)
pair[0].append(30)
print(pair)
```

QUESTIONS

1. Predict the exact output.
2. Explain why the caller's list changes but the caller's integer binding does not.
3. Why can a list inside a tuple change? What happens with `pair[0] = [99]`?

COMPACT ANSWERS

1. Exact output:

```
[1, 2, 3] 0 1
([10, 30], 20)
```

2. For a list, `+=` extends the same object, which numbers also references. Integers are immutable: `count += 1` binds the local `count` to a new value. The caller's `count` stays 0, while `returned` receives 1.
3. The tuple holds fixed references, including a reference to a mutable list. `append` changes that list without replacing the tuple entry. `pair[0] = [99]` tries to replace an entry and raises `TypeError`.

28. Equality versus identity: == and is

PREDICT OUTPUT + EXPLAIN

```
a = [1, 2]
b = [1, 2]
c = a

print(a == b, a is b)
print(a == c, a is c)
c.append(3)
print(a == b, a is c)
```

QUESTIONS

1. Predict the exact output and explain each comparison.
2. Which operator should compare collection contents, and which should check for None?
3. Why should code not rely on is to compare ordinary strings or integers by value?

COMPACT ANSWERS

1. Exact output:

```
True False
True True
False True
```

a and b initially have equal contents but are distinct lists. c is an alias of a, so both comparisons are True. c.append(3) changes a to [1, 2, 3]; it now differs from b, while a and c remain the same object.

2. Use == for collection contents. Use is None (or is not None) to check the singleton None.
3. is tests object identity, not value equality. Python may cache integers or intern strings; reuse is not a reliable value-comparison rule. Use == for ordinary string and integer values.

29. Shallow copy versus deep copy: nested references

PREDICT OUTPUT + EXPLAIN

```
import copy
original = {"scores": [10, 20]}
shallow = copy.copy(original)
deep = copy.deepcopy(original)

shallow["scores"].append(30)
shallow["label"] = "updated"
deep["scores"].append(40)

print(original)
print(shallow)
print(deep)
```

QUESTIONS

1. Predict the exact output.
2. Explain which dictionaries and nested lists are shared.
3. Why does adding label to shallow not add it to original, while appending to shallow['scores'] affects original?
4. How would plain assignment differ from both copy operations?

COMPACT ANSWERS

1. Exact output:

```
{'scores': [10, 20, 30]}
{'scores': [10, 20, 30], 'label': 'updated'}
{'scores': [10, 20, 40]}
```

2. All three outer dictionaries are distinct. original and shallow reference the same scores list. deep has a distinct copied scores list.
3. Adding label changes the shallow dictionary alone. Appending 30 mutates the shared nested list, so both original and shallow show it. Appending 40 affects only deep's list.
4. alias = original creates no copy: both names reference one dictionary. Adding a key or mutating its nested list through either name is visible through the other.

30. Lambda functions: closure capture and late binding

PREDICT OUTPUT + FIX

```
functions = [lambda: i * 2 for i in range(3)]  
print([function() for function in functions])
```

QUESTIONS

1. Predict the exact output and explain when the variable `i` is read.
2. Fix the lambdas so they remember the value of `i` from their own creation and produce the intended doubled values.
3. Explain what a lambda function is, its expression-versus-statement restriction, and one useful application as a key function.

COMPACT ANSWERS

1. Output: `[4, 4, 4]`. The lambdas share the comprehension's `i` binding and read it when called. The loop has finished, so `i` is 2 for all three calls.
2. Capture each current value as a default argument, evaluated when that lambda is created:

```
functions = [lambda i=i: i * 2 for i in range(3)]  
print([function() for function in functions]) # [0, 2, 4]
```

3. lambda creates an anonymous function with one expression whose value is returned automatically. It cannot contain a statement block such as a for statement, try block, or assignment statement. Useful for a short sorting key:

```
players = [{"name": "Jay", "score": 90},  
           {"name": "Sam", "score": 70}]  
ordered = sorted(players, key=lambda player: player["score"])  
# Sam (70) comes before Jay (90).
```